Приложения

Код решения всех задач

Приложение 1

```
library("lmtest")

data = swiss


#1

mean(data$Agriculture)

mean(data$Examination)

mean(data$Infant.Mortality)

var(data$Agriculture)

var(data$Examination)

var(data$Infant.Mortality)

sd(data$Agriculture)

sd(data$Examination)

sd(data$Infant.Mortality)
```

#2 Построим зависимости *Agriculture~Examination* и *Agriculture~Infant.Mortality*

```
model1 = lm(Agriculture~Examination, data)

summary(model1)


model2 = lm(Agriculture~Infant.Mortality, data)

summary(model2)
```

Приложение 2

```
library("lmtest")

data(swiss)

#проверяем отсутствие линейной зависимости
model1 = lm(Fertility~Catholic+Infant.Mortality, data)
model2 = lm(Catholic~Fertility+Infant.Mortality, data)
model3 = lm(Infant.Mortality~Fertility+Catholic, data)
summary(model1)
summary(model2)
summary(model3)

#зависимости нет

model4 = lm(Examination~Fertility+Catholic+Infant.Mortality, data)
summary(model4)

#заметим, что Infant.Mortality не значим, разница без него и с ним менее 5%

#с логарифмами

model5 = lm(Examination~log(Fertility)+Catholic, data)
summary(model5)

model6 = lm(Examination~Fertility+log(Catholic), data)
summary(model6)
```

```
model7 = lm(Examination~Fertility+Catholic+log(Infant.Mortality), data)
summary(model7)

#сравниваем модели, выбираем лучшую

model8 = lm(Examination~I(Fertility*Fertility)+Fertility+Catholic, data)
summary(model8)

#нет связи между I(Fertility*Fertility) и Examination

model80 = lm(I(Fertility*Fertility)~Catholic+Infant.Mortality, data)
summary(model80)

model9 = lm(Examination~Fertility+Catholic+I(Catholic*Catholic), data)
summary(model9)

model90 = lm(I(Catholic*Catholic)~Fertility+Infant.Mortality, data)
summary(model90)

#можно убрать Catholic из 9 модели, однако модель все еще плоха, так как
есть зависимость между регрессорами

model10 =
lm(Examination~Fertility+Catholic+I(Infant.Mortality*Infant.Mortality), data)
summary(model10)

model100 = lm(I(Infant.Mortality*Infant.Mortality)~Fertility+Catholic, data)
summary(model100)
```

```
#нет зависимости

model11 =
lm(Examination~I(Fertility*Catholic)+Fertility+Catholic+Infant.Mortality, data)

summary(model11)

model110 = lm(I(Fertility*Catholic)~Fertility+Catholic+Infant.Mortality, data)

summary(model110)

#плохая модель, есть связь между регрессорами

model12 = lm(Examination~I(Fertility*Infant.Mortality)+Fertility+Catholic, data)

summary(model12)

model120 = lm(I(Fertility*Infant.Mortality)~Fertility+Catholic+Infant.Mortality,
data)

summary(model120)

#плохая модель, есть связь между регрессорами и нет связи с
дополнительной функцией в 12 модели

model13 = lm(Examination~Fertility+I(Infant.Mortality*Catholic), data)

summary(model13)

model130 = lm(I(Infant.Mortality*Catholic)~Fertility, data)

summary(model130)
```

#плохая модель, есть связь между регрессорами с Catholic, если убрать данный регрессор, то появляется неплохая модель

#Находим лучшие 5, 13

Приложение 2.2

```
library("lmtest")

data(swiss)

model4 = lm(Examination~Fertility+Catholic+Infant.Mortality, data)
summary(model4)

#43 степени свободы 4, рассчитано 4 коэффициента, 43-4=39

t_critical = qt(0.975, df = 39) #2.022691

#находим доверительные интервалы для каждого регрессора:
# b=[-0.36-2.02*0.08, -0.36+2.02*0.08] Fertility [-0.5216, -0.1984] нет 0
# b=[-0.07-2.02*0.02, -0.07+2.02*0.02] Catholic [-0.1104, -0.0296] нет 0
# b=[0.5-2.02*0.31, 0.5+2.02*0.31] Infant.Mortality [-0.1262, 1.1262] есть 0 =>
регрессор плохо связан с объясняемой переменной

#доверительный интервал для одного прогноза

new.data = data.frame(Catholic = 20, Fertility = 10, Infant.Mortality = 10)
predict(model4, new.data, interval = "confidence")
```

Приложение 3

```
library("lmtest")

library("rlms")

library("dplyr")

library("GGally")

library(car)

library(sandwich)


data <-
read.csv("C:\\Users\\79286\\Desktop\\Университет\\НИР\\r21i_os26c.csv")

View(data)

data2 = select(data, qj13.2, qh5, q_marst, q_educ, q_age, status, qj6.2,  qj1.1.1,
qj1.1.2, qj1.1.3)

#зарплата, пол, семейное положение, наличие высшего образования, возраст,

#тип населенного пункта, длительность рабочей недели

#Насколько Вы удовлетворены или не удовлетворены Вашей работой в
целом?

#Насколько Вы удовлетворены или не удовлетворены условиями Вашего
труда?

#Насколько Вы удовлетворены или не удовлетворены оплатой Вашего труда?

View(data2)

data2$qj13.2[which(data2$qj13.2>99999990)] <- NA

data2$qj6.2[which(data2$qj6.2>99999990)] <- NA

data2$q_marst[which(data2$q_marst>99999990)] <- NA


##избавляемся от NA

data2 = na.omit(data2)


##нормализуем переменные, имеющие множестово значений
```

```
#зарплата
sal1 = as.character(data2$qj13.2)#преобразование в строку
sal2 = lapply(sal1, as.integer)#получаем список
sal = as.numeric(unlist(sal2))#преобразуем список в вектор и преобразуем этот вектор символов в числовой вектор
mean(sal)
data2["salary"] = (sal - mean(sal)) / sqrt(var(sal))
#data2["salary"]

#возраст
age1 = as.character(data2$q_age)
age2 = lapply(age1, as.integer)
age3 = as.numeric(unlist(age2))
data2["age"]= (age3 - mean(age3)) / sqrt(var(age3))
#data2["age"]

#длительность рабочей недели
dur1 = as.character(data2$qj6.2)
dur2 = lapply(dur1, as.integer)
dur3 = as.numeric(unlist(dur2))
data2["dur"] = (dur3 - mean(dur3)) / sqrt(var(dur3))

##создаем новые переменные
#пол
data2["sex"]=data2$qh5
data2["sex"] = lapply(data2["sex"], as.character)
data2$sex[which(data2$sex!='1')] <- 0
data2$sex[which(data2$sex=='1')] <- 1
```

```
data2$sex = as.numeric(data2$sex)

#наличие высшего образования
data2["h_educ"] = data2$q_educ
data2["h_educ"] = lapply(data2["h_educ"], as.character)
data2["higher_educ"] = data2$u_educ
data2["higher_educ"] = 0
data2$higher_educ[which(data2$h_educ=='21')] <- 1
data2$higher_educ[which(data2$h_educ=='22')] <- 1
data2$higher_educ[which(data2$h_educ=='23')] <- 1

#населенный пункт
data2["status1"]=data2$status
data2["status1"] = lapply(data2["status1"], as.character)
data2["status2"] = 0
data2$status2[which(data2$status1=='1')] <- 1
data2$status2[which(data2$status1=='2')] <- 1
data2$status2 = as.numeric(data2$status2)

#семейное положение
data2["wed"]= data2$q_marst
data2["wed"] = lapply(data2["wed"], as.character)
data2$wed1 = 0
data2$wed1[which(data2$wed=='1')] <- 1
data2$wed1[which(data2$wed=='3')] <- 1
data2$wed1 = as.numeric(data2$wed1)

data2["wed2"] = lapply(data2["wed"], as.character)
```

```
data2$wed2 = 0
data2$wed2[which(data2$wed=='2')] <- 1
data2$wed2 = as.numeric(data2$wed2)

data2["wed3"]=data2$q_marst
data2$wed3 = 0
data2$wed3[which(data2$wed=='4')] <- 1
data2$wed3 = as.numeric(data2$wed3)

data2["wed4"]=data2$q_marst
data2$wed4 = 0
data2$wed4[which(data2$wed=='5')] <- 1
data2$wed4 = as.numeric(data2$wed4)

#удовлетворенность
data2["sat1"]=data2$qj1.1.1
data2["sat1"] = lapply(data2["sat1"], as.character)
data2["satisfy1"] = 0
data2$satisfy1[which(data2$sat1=='1')] <- 1
data2$satisfy1[which(data2$sat1=='2')] <- 1
data2$satisfy1 = as.numeric(data2$satisfy1)

data2["sat2"]=data2$qj1.1.2
data2["sat2"] = lapply(data2["sat2"], as.character)
data2["satisfy2"] = 0
data2$satisfy2[which(data2$sat2=='1')] <- 1
data2$satisfy2[which(data2$sat2=='2')] <- 1
data2$satisfy2 = as.numeric(data2$satisfy2)
```

```
data2["sat3"]=data2$qj1.1.3

data2["sat3"] = lapply(data2["sat3"], as.character)

data2["satisfy3"] = 0

data2$satisfy3[which(data2$sat3=='1')] <- 1

data2$satisfy3[which(data2$sat3=='2')] <- 1

data2$satisfy3 = as.numeric(data2$satisfy3)


data2["satisfaction"] = data2$satisfy1 * data2$satisfy2 * data2$satisfy3


data2 = na.omit(data2)


data3 = select(data2, salary, age, sex, higher_educ, status2, dur,
wed1,wed2,wed3,wed4, satisfaction)


View(data3)


model1 = lm(data = data3, salary~age + sex + higher_educ + status2 + dur + wed1
+ wed2 + wed3 + wed4 + satisfaction)

summary(model1)

vif(model1)


##############################


model2 = lm(data = data3, salary~log(age) + sex + higher_educ + status2 + dur +
wed1 + wed2 + wed3 + satisfaction)

summary(model2)

vif(model2)
```

```
model3 = lm(data = data3, salary~log(age) + sex + higher_educ + status2 + dur + I(dur^2) + wed1 + wed2 + wed3 + satisfaction)

summary(model3)

vif(model3)


model4 = lm(data = data3, salary~log(age) + sex + higher_educ + status2 + dur + I(dur^2) + I(dur*age) + wed1 + wed2 + wed3 + satisfaction)

summary(model4)

vif(model4)


model5 = lm(data = data3, log(salary)~log(age) + sex + higher_educ + status2 + dur + I(dur^2) + wed1 + wed2 + wed3 + satisfaction)

summary(model5)

vif(model5)


model6 = lm(data = data3, salary~age + higher_educ + status2 + dur + satisfaction)

summary(model6)

vif(model6)


#из вышеперечисленных моделей можем сделать несколько выводов касательно того, у кого выше зарплата

#во-первых, при увелечении возраста, возрастает и зарплата

#положительный коэффициент перед регрессором пола показывает, что у мужчин зарплаты выше

#заметим, что удовлетворенность трудом также сильно влияет на зарплату

#люди с высшим образованием получают более высокую зарплату

#first test ########## 2marst not2status

data2["wedtest1"]= data2$q_marst

data2["wedtest1"] = lapply(data2["wedtest1"], as.character)
```

```
data2$wedtest11 = 0
data2$wedtest11[which(data2$wedtest1=='2')] <- 1
data2$wedtest11 = as.numeric(data2$wedtest11)

data2["statustest1"]=data2$status
data2["statustest1"] = lapply(data2["statustest1"], as.character)
data2["statustest2"] = 0
data2$statustest2[which(data2$statustest1!='2')] <- 1
data2$statustest2 = as.numeric(data2$statustest2)

#second test ########## #1,4marst 21-23educ
data2["wedtest2"]= data2$q_marst
data2["wedtest2"] = lapply(data2["wedtest2"], as.character)
data2$wedtest22 = 0
data2$wedtest22[which(data2$wedtest2=='1')] <- 1
data2$wedtest22[which(data2$wedtest2=='4')] <- 1
data2$wedtest22 = as.numeric(data2$wedtest22)

data4 = select(data2, salary, wedtest11, statustest2, wedtest22, higher_educ)

data4$wedtest22[which(data4$wedtest22>99999990)] <- NA
data4$wedtest11[which(data4$wedtest11>99999990)] <- NA
data4$statustest2[which(data4$statustest2>99999990)] <- NA

##избавляемся от NA
data4 = na.omit(data4)

View(data4)
```

```
model7 = lm(data = data4, salary~ wedtest11 + statustest2)
summary(model7)
vif(model6)

model8 = lm(data = data4, salary~ wedtest22 + higher_educ)
summary(model8)
vif(model6)
```

Приложение 4.1

```
import pandas

import numpy as np

import io

from sklearn.linear_model import SGDClassifier

from sklearn.linear_model import LogisticRegression

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn import svm

from sklearn.ensemble import RandomForestClassifier

data = pandas.read_csv('C:\\Users\\79286\\Desktop\\aug_test.csv',
index_col='enrollee_id')

data_sel = data.loc[:, data.columns.isin(['city_development_index', 'gender',
'relevent_experience', 'enrolled_university', 'education_level', 'major_discipline',
'experience', 'company_size', 'last_new_job', 'training_hours'])]

data_sel = data_sel.dropna()

data_sel['gender']=np.where(data_sel['gender'] == 'other', np.nan,
data_sel['gender'])

data_sel = data_sel.dropna()

data_sel['gender'] = np.where(data_sel['gender'] == 'Male', 0, 1)

data_sel['relevent_experience']=np.where(data_sel['relevent_experience']=='Has
relevent experience', 1, 0)

data_sel['enrolled_university']=np.where(data_sel['enrolled_university']=='no_enro
llment', 0, 1)

data_sel['education_level']=np.where(data_sel['education_level']=='Graduate', 1,
data_sel['education_level'])

data_sel['education_level']=np.where(data_sel['education_level']=='High School',
0, data_sel['education_level'])

data_sel['education_level']=np.where(data_sel['education_level']=='Primary
School', 0, data_sel['education_level'])
```

```
data_sel['education_level']=np.where(data_sel['education_level']=='Masters', 1, data_sel['education_level'])

data_sel['education_level']=np.where(data_sel['education_level']=='Phd', 1, data_sel['education_level'])

data_sel['education_level']=data_sel['education_level'].astype(str).astype(int)

data_sel.pop('education_level')

data_sel['major_discipline'] = np.where(data_sel['major_discipline'] == 'STEM', 1, data_sel['major_discipline'])

data_sel['major_discipline'] = np.where(data_sel['major_discipline'] == 'Business Degree', 1, data_sel['major_discipline'])

data_sel['major_discipline'] = np.where(data_sel['major_discipline'] != 1, 0, data_sel['major_discipline'])

data_sel['major_discipline']=data_sel['major_discipline'].astype(str).astype(int)

data_sel['experience']=np.where(data_sel['experience'] == '>20', 21, data_sel['experience'])

data_sel['experience']=np.where(data_sel['experience'] == '<1', 0, data_sel['experience'])

data_sel['experience']=data_sel['experience'].astype(str).astype(int)

data_sel['company_size']=np.where(data_sel['company_size'] == '10000+', 1, 0)

data_sel['last_new_job']=np.where(data_sel['last_new_job'] == '>4', 5, data_sel['last_new_job'])

data_sel['last_new_job']=np.where(data_sel['last_new_job'] == 'never', 0, data_sel['last_new_job'])

data_sel['last_new_job']=data_sel['last_new_job'].astype(str).astype(int)

gender = data_sel.loc[:, data_sel.columns.isin(['gender'])]

X = data_sel.loc[:, data_sel.columns.isin(['city_development_index', 'relevent_experience', 'enrolled_university', 'education_level', 'major_discipline', 'experience', 'company_size', 'last_new_job', 'training_hours'])]

X_train, X_test, y_train, y_test = train_test_split(X, gender, test_size=0.05)


clf = SGDClassifier(loss="hinge", penalty="l2")

clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)

clf.score(X_test, y_test)


from sklearn.model_selection import cross_val_score

print("accuracy:"+str(np.average(cross_val_score(clf, X_test, y_test,
scoring='accuracy'))))

print("f1:"+str(np.average(cross_val_score(clf, X_test, y_test, scoring='f1'))))

print("precision:"+str(np.average(cross_val_score(clf, X_test, y_test,
scoring='precision'))))

print("recall:"+str(np.average(cross_val_score(clf, X_test, y_test,
scoring='recall'))))


from sklearn.model_selection import GridSearchCV

param_grid = {'n_estimators':[100],'max_features':['auto'],'max_depth':
list(range(1,20))}

forest = GridSearchCV(estimator = RandomForestClassifier(),param_grid =
param_grid,cv = 5, refit = True)

forest.fit(X_train, y_train)


data_sel.corr() #нет сильной корреляции
```

Приложение 4.2

```
import pandas

import numpy as np

import io

from sklearn.linear_model import LogisticRegression

from sklearn import datasets

from sklearn.model_selection import train_test_split

data =
pandas.read_csv("C:\\Users\\79286\\Desktop\\Университет\\НИР\\kc_house_data.
csv", index_col = 'id')

data


data.info()


data.isnull().sum()


data["date"].value_counts()


data_sel = data.loc[:,data.columns.isin(['price', 'bedrooms',
'bathrooms','sqft_living','sqft_lot','floors','waterfront','view','condition','grade','sqft_
above','sqft_basement','yr_renovated','yr_built','lat','long','sqft_living15','sqft_lot15'
])]

data_sel = data_sel.dropna()

data_sel['bedrooms'] = np.where(data['bedrooms'] <= 3, 0, 1)

data_sel['view'] = np.where(data['view'] == 0, 0, 1)

data_sel['condition'] = np.where(data['condition'] >= 4, 1, 0)

data_sel['yr_renovated'] = np.where(data['yr_renovated'] > 100, 1, 0)

data_sel['sqft_lot15'] = np.where(data['sqft_lot15'] > 10000, 1, 0)

Bedrooms = data_sel.loc[:,data_sel.columns.isin(['bedrooms'])]
```

```
X = data_sel.loc[:,data_sel.columns.isin(['price',
'bathrooms','sqft_living','sqft_lot','floors','waterfront','view','condition','grade','sqft_
above','sqft_basement','yr_renovated','yr_built','lat','long','sqft_living15','sqft_lot15'
])]

x_train, x_test, y_train, y_test = train_test_split(X, Bedrooms,test_size = 0.3)

#y_train = np.ravel(y_train)


classifier = LogisticRegression(random_state = 0, solver = 'lbfgs', multi_class =
'multinomial', penalty = 'l2')#, penalty = 'elasticnet', l1_ration = '0.5' #'saga'

classifier.fit(x_train, y_train)


y_pred = classifier.predict(x_test)

classifier.score(x_test, y_test)


print(len(x_test))

print(len(y_test))

print(sum(y_test['bedrooms']))

print(sum(y_pred))

print(np.where(y_test == 1)[0])


TP = 0

FP = 0

FN = 0

TN = 0

for i in np.where(y_test == 1)[0]:

    if y_pred[i] == 1:

        TP+=1

    else:

        FN+=1
```

```
for i in np.where(y_test == 0)[0]:
    if y_pred[i] == 1:
        FP+=1
    else:
        TN+=1
print(TP,FP,FN, TN)
precision = TP / (TP + FP)
recall = TP/(TP + FN)
F1 = 2* (precision*recall)/(precision+recall)
print(precision, recall,F1)


from sklearn.model_selection import cross_val_score
print("accuracy:"+str(np.average(cross_val_score(classifier, x_test, y_test,
scoring='accuracy'))))
print("f1:"+str(np.average(cross_val_score(classifier, x_test, y_test, scoring='f1'))))
print("precision:"+str(np.average(cross_val_score(classifier, x_test, y_test,
scoring='precision'))))
print("recall:"+str(np.average(cross_val_score(classifier, x_test, y_test,
scoring='recall'))))


from sklearn.ensemble import RandomForestClassifier
forest = RandomForestClassifier(n_estimators = 210)
print("accuracy:"+str(np.average(cross_val_score(forest, x_test, y_test,
scoring='accuracy'))))
print("f1:"+str(np.average(cross_val_score(forest, x_test, y_test, scoring='f1'))))
print("precision:"+str(np.average(cross_val_score(forest, x_test, y_test,
scoring='precision'))))
print("recall:"+str(np.average(cross_val_score(forest, x_test, y_test,
scoring='recall'))))
```

```
from sklearn.model_selection import GridSearchCV

param_grid =
{'n_estimators':[100,150,200,250,300],'max_features':['auto'],'max_depth':
list(range(1,20))}

forest = GridSearchCV(estimator = RandomForestClassifier(),param_grid =
param_grid,cv = 5, refit = True)

forest.fit(x_train, y_train)


forest.best_estimator_

y_pred = forest.best_estimator_.predict(x_test)

print(y_pred)


print("accuracy:"+str(np.average(cross_val_score(forest.best_estimator_, x_test,
y_test, scoring='accuracy'))))

print("f1:"+str(np.average(cross_val_score(forest.best_estimator_, x_test, y_test,
scoring='f1'))))

print("precision:"+str(np.average(cross_val_score(forest.best_estimator_, x_test,
y_test, scoring='precision'))))

print("recall:"+str(np.average(cross_val_score(forest.best_estimator_, x_test,
y_test, scoring='recall'))))


forest.best_estimator_

param_grid = {'n_estimators':list(range(100, 351,
10)),'max_features':['auto'],'max_depth': list(range(1,20)),'criterion': ['gini']}

forest = GridSearchCV(estimator = RandomForestClassifier(),param_grid =
param_grid,cv = 5, refit = True)

forest.fit(x_train, y_train)


forest.best_estimator_
```

```
print("accuracy:"+str(np.average(cross_val_score(forest.best_estimator_, x_test, y_test, scoring='accuracy'))))

print("f1:"+str(np.average(cross_val_score(forest.best_estimator_, x_test, y_test, scoring='f1'))))

print("precision:"+str(np.average(cross_val_score(forest.best_estimator_, x_test, y_test, scoring='precision'))))

print("recall:"+str(np.average(cross_val_score(forest.best_estimator_, x_test, y_test, scoring='recall'))))
```

Приложение 5-6

```
#include <iostream>
#include <vector>
#include <ctime>

class Combinations {
private:
    unsigned long long M, N;
public:
    Combinations(unsigned long long M, unsigned long long N) : M(M), N(N) {}

    //вычисление числа сочетаний
    int count() const {
        if (M > N)
            return 0;
        int res = 1;
        for (int i = 1; i <= M; ++i) {
            res *= N - i + 1;
            res /= i;
        }
        return res;
    }

    //генерация всех сочетаний
    std::vector<std::vector<int>> generate() const {
        std::vector<std::vector<int>> res;
        std::vector<int> cur(M);
        for (int i = 0; i < M; ++i)
            cur[i] = i;
        while (true) {
            res.push_back(cur);
            int j = M - 1;
            while (j >= 0 && cur[j] == N - M + j)
                --j;
            if (j < 0)
                break;
            ++cur[j];
            for (int k = j + 1; k < M; ++k)
                cur[k] = cur[j] + k - j;
        }
        return res;
    }
};
```

```
int main() {
    int start = clock();
    Combinations c(3, 5);
    std::cout << "Number of combinations: " << c.count() << "\n";
    std::vector<std::vector<int>> combs = c.generate();
    for (const auto& comb : combs) {
        for (int x : comb)
            std::cout << x << " ";
        std::cout << "\n";
    }
    int end = clock();
    int t = (end - start) / CLOCKS_PER_SEC;
    std::cout << t;
    return 0;
}
```